

RV UNIVERSITY, BENGALURU-59

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



A Project Report On

bytemingos Campus Food Ordering Application

Submitted in Fulfillment for the award of degree of

BSc(Honors)

In

School of Computer Science and Engineering

Submitted By

Name: KUNAL KAJREKAR USN: 1RUA24SCS0053

Under the Guidance of

Prof. Shrishti Kumari

School of CSE

RV University, Bengaluru-560059

2025-2026

Abstract

bytemingos is a full stack campus food ordering application developed using React.js, Vite, Node.js and Express.js. Customers can browse dishes, change their cart, choose a pickup counter and follow the status of an order. A kitchen dashboard updates orders through four sequential stages, while customers receive the updated status through periodic API requests.

The project implements the React concepts specified in CS3301 CIE2, including functional components, a class component, props, parent-child communication, state, effects, event handling, controlled forms and client-side routing. Two additional features personalise the food-ordering workflow: an explainable meal finder and persistent favourite meals. The interface uses plain CSS with responsive layouts and a tomato-and-cream visual identity.

The production build and backend integration tests passed. Browser verification covered menu filtering, cart persistence, checkout, kitchen updates, customer receipts and mobile layouts. The result is a working local ordering flow with sample dishes and payment at the pickup counter.

Table of Contents

Introduction	4
Problem Statement and Objectives	5
Technologies Used	6
Selected YouTube Tutorial	7
System and Component Structure	8
Shared Components and State Functions	9
Menu Cart and Checkout Components	10
Favourite Recommendation and Receipt Components	11
Kitchen Backend and Supporting Functions	12
Important React Concepts Implemented	13
Screenshots	14–17
Modifications Made	18
Challenges Faced and Verification	19
Conclusion	20
GitHub Links and References	21

Introduction

bytemingos brings the main steps of a campus pickup order into one website. Customers browse food, choose quantities, save favourites and submit pickup details. The kitchen views the received orders and records their preparation progress.

- Customer workflow: Menu selection → lunch bag → pickup checkout → order receipt → collection.
- Kitchen workflow: Order received → preparing → ready for pickup → collected.
- Frontend responsibility: Display the menu, manage interactive state, validate forms and navigate between views.
- Backend responsibility: Validate order data, calculate authoritative prices, persist records and control status changes.

The selected GreatStack tutorial was the starting reference for the food-ordering flow. The project uses a different campus-pickup interface and a smaller Express backend suited to this mini project.

Problem Statement and Objectives

A static food menu cannot maintain selected quantities, record pickup details or communicate order readiness. Students with short breaks need a clear way to select meals and see the progress of their order. A React interface and Express backend make these actions possible.

Project Objectives

- Divide the interface into reusable components with clearly defined responsibilities.
- Use props and callbacks to demonstrate parent-child data flow.
- Manage user choices and forms with `useState`; handle data loading, persistence and polling with `useEffect`.
- Provide multiple views using React Router and demonstrate a class-based error boundary.
- Implement Express endpoints for menu retrieval, recommendations, checkout, receipts and kitchen updates.
- Add two features beyond the tutorial workflow: preference-based meal recommendations and saved favourite meals.
- Provide a responsive interface and verify the complete customer-to-kitchen ordering flow.

Project Scope

The application contains twelve sample dishes, three pickup counters and four order stages. Orders are persisted in a local JSON file. Payment is recorded as pay at pickup; online payment, real delivery, SMS and customer accounts are outside the implemented scope.

Technologies Used

- **React.js:** Builds functional UI components and the class-based `ErrorBoundary`.
- **JavaScript ES6+:** Uses modules, destructuring, arrow functions, array methods, promises and `async/await`.
- **HTML5:** Supplies semantic sections, headings, navigation and native form controls.
- **CSS3:** Defines the colour palette, responsive grid/flex layouts, card styling and keyboard focus states.
- **React Router:** Maps URLs to pages and supports parameterised order-receipt routes.
- **Vite:** Runs the development frontend and generates the production asset bundle.
- **Node.js and Express.js:** Execute the backend and define HTTP routes, validation and middleware.
- **CORS:** Permits requests from the configured Vite frontend during development.
- **Fetch API:** Sends JSON requests from React and handles API responses.
- **React Context and localStorage:** Share application state and preserve device-local cart, favourites and receipt keys.
- **JSON and Node file APIs:** Store the sample catalogue and persist orders through queued, atomic file writes.
- **Git and GitHub:** Support source version control and repository sharing. The source is tracked with Git and shared through the public BYTEMINGOS repository.
- **Lucide React:** Supplies consistent interface icons for cart, favourites, location and order status.

Selected YouTube Tutorial

- Channel: GreatStack.
- Tutorial: How To Create Full Stack Food Delivery Website In React JS, MongoDB, Express, Node JS & Stripe.
- Tutorial URL: <https://www.youtube.com/watch?v=DBMPXJJfQEA>

The tutorial provides a suitable reference for food cards, cart interaction and customer/admin ordering workflows. bytemingos retains these core ideas while changing the visual identity and adapting checkout to campus pickup.

Project Adaptations

- Plain CSS replaces utility-based styling, allowing the responsive layout to be explained directly.
- A local JSON store replaces MongoDB for the implemented mini-project backend.
- Pay-at-pickup checkout replaces online payment integration.
- MealFinder and Saved introduce preference-based meal matching and persistent favourites.

The food photographs are illustrative assets from a community implementation of the tutorial. Menu descriptions, prices and ingredient labels are sample data created for the project.

System and Component Structure

React handles the pages and user choices. Express checks incoming data and stores the orders. The two parts communicate through JSON requests.

- App entry point: `main.jsx` mounts App, the router, the shared state provider and the error boundary.
- Page layout: Layout wraps the route-specific pages and keeps navigation consistent.
- Reusable controls: `FoodCard`, `Quantity`, `DishGrid` and `Summary` serve multiple views.
- Shared state: `StoreProvider` owns the catalogue, cart, favourites, receipt keys and feedback state.
- API requests: `api.js` centralises Fetch requests, response parsing, currency formatting and stage labels.
- Backend: `createApp` configures Express endpoints, middleware and the order store.
- Stored data: `menu.json` supplies the catalogue; `orders.json` is created when an order is saved.

Data Flow

- Menu → DishGrid → FoodCard: Dish data and event callbacks pass from parent to child through props.
- Quantity → parent callback → StoreProvider: A quantity event changes shared state and updates totals across views.
- Checkout → POST `/api/orders` → `saveReceipt`: The server returns the persisted order, and the frontend stores its receipt key.
- Kitchen → PATCH status → `useReceipt` refresh: The customer receipt reflects the latest server status.

The production frontend is served by Express from the build folder. During development, Vite and Express run separately and communicate through the configured API origin.

Shared Components and State Functions

App and ScrollToTop in main.jsx

- `App()`: Composes `ErrorBoundary`, `BrowserRouter`, `StoreProvider` and the route tree. The fallback route provides recovery from unknown URLs.
- `ScrollToTop()`: Reads `pathname` with `useLocation` and scrolls to the top when a route changes.

Layout in components/Layout.jsx

- `Layout()`: Renders the shared brand, navigation, cart count and footer. `Outlet` places the selected page inside this shell; `NavLink` identifies the active view.

ErrorBoundary in components/ErrorBoundary.jsx

- `getDerivedStateFromError()`: Changes the class state so a rendering failure displays a recovery screen.
- `componentDidCatch()`: Records the caught interface error. `render()` returns either the child components or the reload fallback.

StoreProvider in Store.jsx

- `StoreProvider()`: Holds menu, loading/error state, cart quantities, favourites, receipt keys and toast feedback. It derives cart lines, total portions and subtotal from the current catalogue.
- `readSaved()` and `save()`: Parse and persist `localStorage` values with safe fallbacks. Saved data from the original project name remains accessible after the rebrand.
- `loadMenu()`: Fetches `/api/menu` and updates the loading, error and catalogue states.
- `changeQuantity()`: Applies a portion change, limits each dish to ten portions and removes a dish when its quantity reaches zero.
- `add()` and `toggleFavourite()`: Add a portion with feedback, or toggle a dish ID in the favourites list.
- `saveReceipt()`: Stores the order ID, tracking token and timestamp, retains the latest thirty receipt keys, and clears the submitted cart.
- `useStore()`: Gives child components access to the shared context. Effects save local state and clear toast timers.

Menu Cart and Checkout Components

Menu MenuState and DishGrid in pages/Menu.jsx

- `Menu()`: Stores search, category, diet and sort choices. `useMemo` filters the catalogue and orders dishes by featured status, price or preparation time.
- `MenuState()`: Displays loading feedback or an API error with a retry action. It returns no extra UI once the catalogue is available.
- `DishGrid()`: Maps dish objects into `FoodCard` components and supplies quantities, favourite flags, optional recommendation reasons and callbacks.

FoodCard and Quantity in components/FoodCard.jsx

- `FoodCard()`: Displays a dish photograph, description, price, diet, preparation time and listed ingredients. Its props connect add/save controls to parent handlers.
- `Quantity()`: Shows decrement, current count and increment controls. `onChange` reports -1 or +1 to the parent; increment is disabled at ten portions.

Cart and Summary in pages/Cart.jsx

- `Cart()`: Displays the selected dish lines, quantity controls and removal actions. It uses shared cart state and shows a recovery path when the bag is empty.
- `Summary()`: Displays portion count, subtotal, free pickup and total. Its button prop controls whether the checkout navigation action is rendered, so the same summary is reused during checkout.

Checkout in pages/Checkout.jsx

- `Checkout()`: Controls name, mobile number, pickup counter and notes. Native form constraints check the required fields before submission.
- `submit()`: Prevents the default form action, sets a busy state and sends customer details with dish IDs and quantities. On success it saves the receipt and navigates to the order page; failures are displayed without clearing the bag.

Favourite Recommendation and Receipt Components

Saved in pages/Saved.jsx

- `Saved()`: Filters the catalogue using the stored favourite IDs. It reuses `DishGrid` for saved dishes and provides an empty state when no favourites exist.

MealFinder in pages/MealFinder.jsx

- `MealFinder()`: Controls budget, diet, maximum preparation time and ingredient exclusions. It displays matching cards, explanations, errors and the no-match state.
- `find()`: Converts numeric preference values, posts them to `/api/recommendations` and stores the returned shortlist. Busy state prevents repeated submissions while the request is in progress.

Orders and Receipt Functions in pages/Orders.jsx

- `useReceipt()`: Requests the order with its stored tracking token. The effect refreshes every eight seconds, tracks loading/errors, prevents stale updates after unmount and clears the interval.
- `OrderRow()`: Summarises one receipt with its ID, portion count, total, timestamp and current status; selecting the row opens the detailed receipt.
- `Orders()`: Lists the receipt keys stored on the device. It renders an empty-order state when no receipt keys are available.
- `OrderDetails()`: Reads the order ID with `useParams`, locates the matching receipt key and displays the fetched order. It renders the four-stage timeline, pickup counter, item totals, payment method, notes and initial preparation estimate.

Receipt keys are device-local. The order record remains on the server, and its ID alone is insufficient to retrieve the receipt.

Kitchen Backend and Supporting Functions

Kitchen in pages/Kitchen.jsx

- `Kitchen()`: Manages the entered PIN, active session PIN, orders, filters and request feedback. Active, Collected and All filters control the displayed order cards.
- `login()`: Requests the kitchen order list using the supplied PIN. A successful response opens the board; a rejected request leaves the login view active.
- `advance()`: Calculates the next stage, submits a PATCH request and refreshes the kitchen list. Polling retrieves new orders every eight seconds; locking the board clears the local session state.

Server Functions in backend/app.js and server.js

- `createApp()`: Loads the menu and saved orders, validates the store structure and creates the configured Express application.
- `commit()`: Serialises changes through a promise queue. It clones the current orders, applies the change, writes a temporary file, replaces the store atomically and only then updates in-memory data.
- Order creation handler: Validates customer fields, distinct dish IDs, integer quantities and notes. It calculates totals from menu prices and generates an order ID and random receipt token.
- Receipt and kitchen handlers: Verify the receipt token or kitchen PIN. Status updates permit only the next stage and re-check it inside the queued write.
- `server.js`: Loads environment settings, starts the local HTTP listener and reports startup errors.

Supporting Files

- `api()`: Sends JSON requests and converts unsuccessful responses into errors for the calling component. `money()` formats INR amounts; `stages` supplies the shared frontend stage labels.
- `menu.json`: Defines the twelve dishes, prices, photographs, categories, diet labels, preparation times and sample ingredient lists.
- `index.html` and `vite.config.js`: Supply the root document, metadata, React build plugin and development server configuration.
- `styles.css`: Implements the visual identity, component layouts, responsive breakpoints, focus states and reduced-motion behaviour.

Important React Concepts Implemented

- **Functional components:** All main pages and reusable food-ordering UI sections are implemented as functions.
- **Class component:** `ErrorBoundary` extends `Component` and uses error-handling lifecycle methods.
- **Props:** Dish objects, quantity values, saved flags and callback functions are passed into `FoodCard` and `Quantity`.
- **Parent-child communication:** Child controls report interactions through callbacks; the parent updates shared state and passes the new values back down.
- **`useState`:** Stores filters, preferences, form values, cart quantities, favourites, PIN state and request feedback.
- **`useEffect`:** Loads the menu, persists local values, refreshes receipts/kitchen orders and cleans up timers.
- **`useMemo`:** Recomputes the filtered and sorted catalogue when menu data or filter choices change.
- **Context:** `StoreProvider` and `useStore` share cart, favourites and receipt-key data across routes.
- **Event handling:** Click events change quantities and favourites; change events control fields; submit events send checkout and preference requests.
- **Form handling:** Checkout combines browser constraints with independent backend validation; `MealFinder` submits controlled preference values.
- **Routing:** `BrowserRouter`, `Routes`, `Route`, `NavLink`, `Link`, `useNavigate` and `useParams` provide page navigation and parameterised receipts.
- **Responsive design:** CSS breakpoints at 1150, 800 and 560 pixels adapt columns, navigation and mobile food-card rows.

Screenshots of the Menu

The following figures show the implemented application connected to its Express backend. Checkout and order figures use sample customer data.

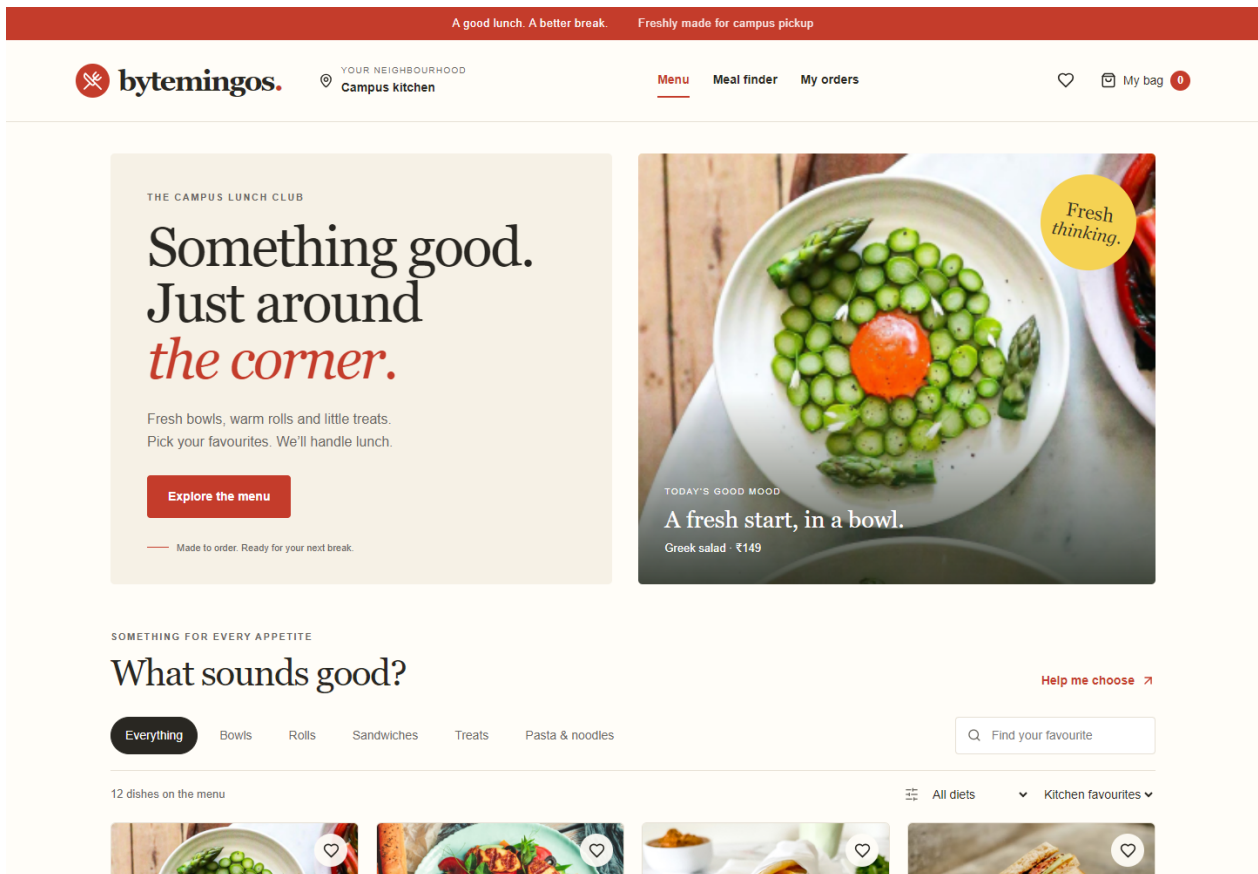
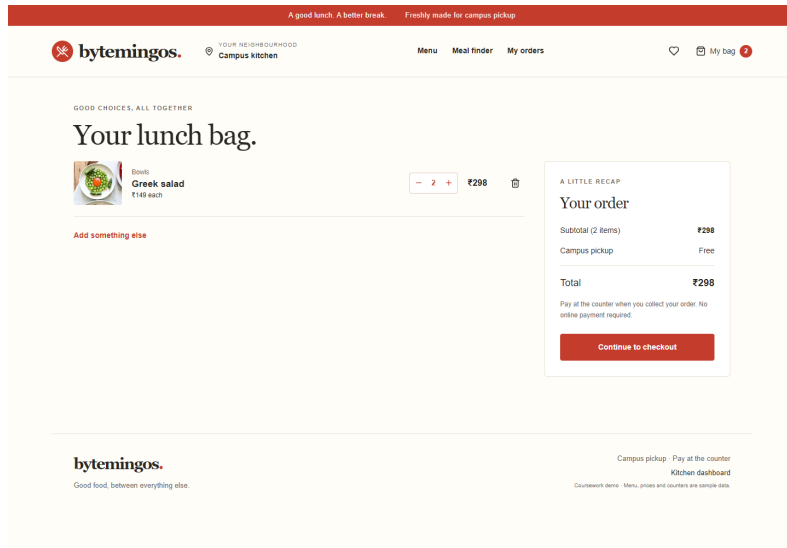


Figure 1. Desktop menu with category navigation, search, dietary filters and food cards.

- The menu presents twelve dishes with price, preparation time, quantity and favourite controls.

Screenshots of Ordering



A good lunch. A better break. Freshly made for campus pickup

bytemingos. YOUR NEIGHBOURHOOD Campus kitchen Menu Meal finder My orders

GOOD CHOICES, ALL TOGETHER

Your lunch bag.

 **Greek salad**
₹149 each

2 ₹298

[Add something else](#)

A LITTLE RECAP

Your order

Subtotal (2 items)	₹298
Campus pickup	Free
Total	₹298

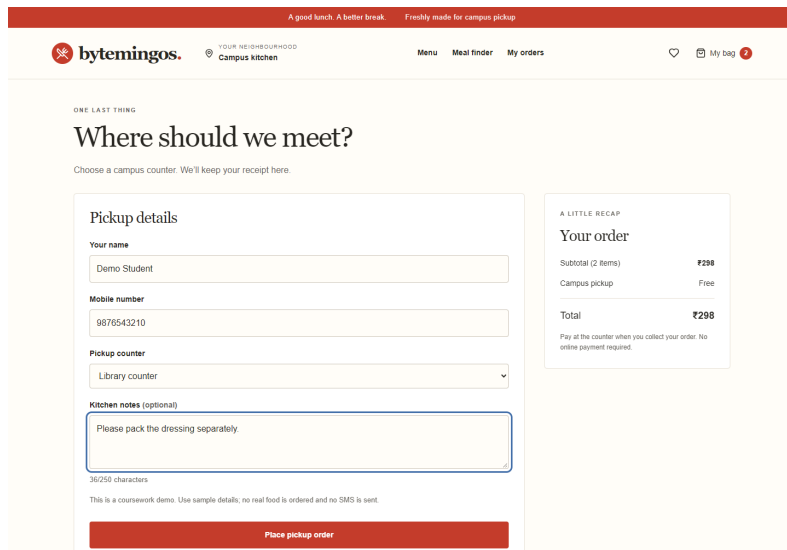
Pay at the counter when you collect your order. No online payment required.

[Continue to checkout](#)

bytemingos.
Good food, between everything else.

Campus pickup: Pay at the counter
Kitchen dashboard
Coursework demo: Menu, prices and counters are sample data.

Figure 2. Two portions of Greek salad produce a subtotal of ₹298.



A good lunch. A better break. Freshly made for campus pickup

bytemingos. YOUR NEIGHBOURHOOD Campus kitchen Menu Meal finder My orders

ONE LAST THING

Where should we meet?

Choose a campus counter. We'll keep your receipt here.

Pickup details

Your name
Demo Student

Mobile number
9876543210

Pickup counter
Library counter

Kitchen notes (optional)
Please pack the dressing separately.

36255 characters
This is a coursework demo. Use sample details; no real food is ordered and no SMS is sent.

[Place pickup order](#)

A LITTLE RECAP

Your order

Subtotal (2 items)	₹298
Campus pickup	Free
Total	₹298

Pay at the counter when you collect your order. No online payment required.

Figure 3. Controlled checkout form with pickup selection and kitchen notes.

- Cart totals respond to quantity changes; checkout records a pay-at-pickup order.

Screenshots of Order Tracking

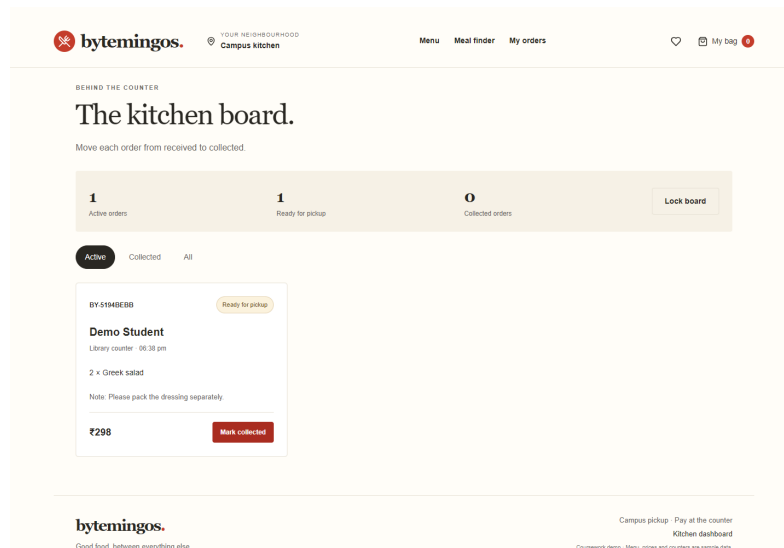


Figure 4. Kitchen order advanced to Ready for pickup.

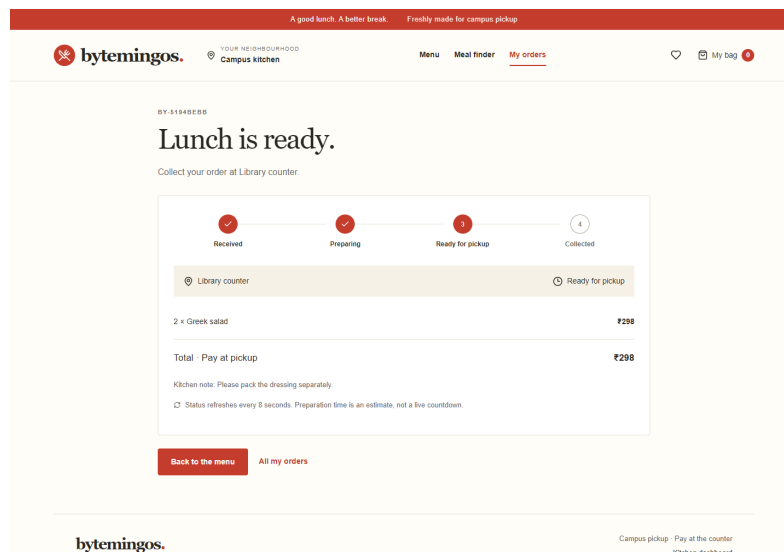


Figure 5. Customer receipt reflects the matching kitchen status.

- The displayed receipt status is fetched from the server rather than simulated in the interface.

Screenshots of Additional Features

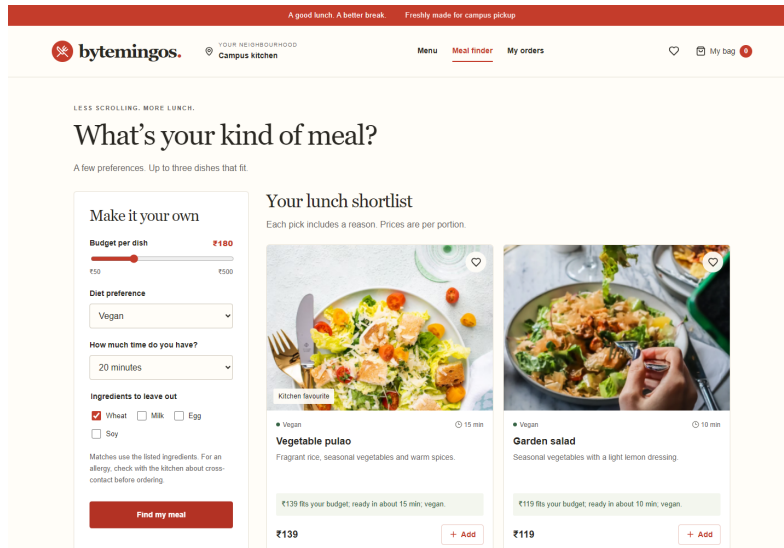


Figure 6. Preference-based meal shortlist with an explanation for each result.

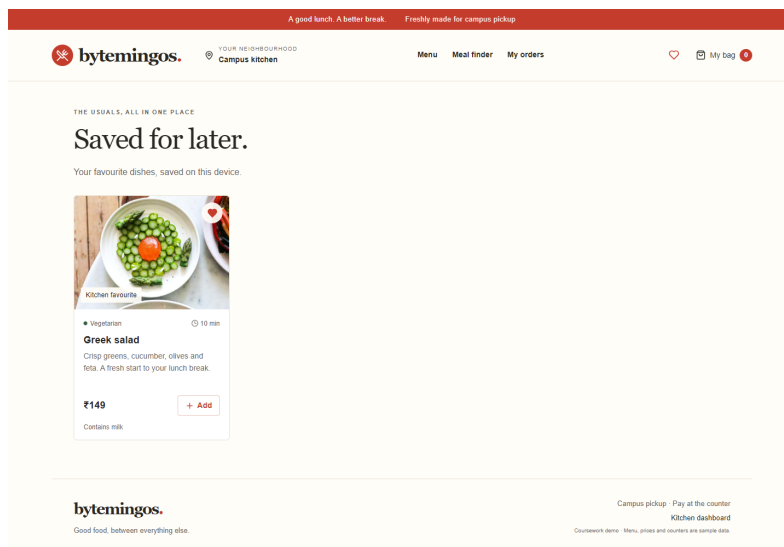


Figure 7. Favourite dish displayed on the dedicated saved-meals page.

- Both additions reuse the food-card interface while providing distinct customer functions.

Modifications Made

Modification One Meal Finder

- Purpose: Reduce menu browsing by finding dishes that fit a customer's budget, diet and available preparation time.
- Frontend implementation: MealFinder provides controlled preference fields and displays results through DishGrid.
- Backend implementation: POST /api/recommendations filters eligible dishes, ranks kitchen favourites first, then lower prices, and returns up to three matches with reasons.
- Diet matching: Vegetarian choices include vegan dishes; other selected diets require a matching label.
- Ingredient exclusions: Dishes listing an excluded ingredient are removed from the shortlist. Empty results produce a clear no-match state.

Modification Two Saved Favourite Meals

- Purpose: Provide quick access to meals a customer wants to order again.
- Frontend implementation: FoodCard's heart invokes toggleFavourite(); Saved filters the catalogue against the stored IDs.
- Persistence: A dedicated effect writes favourite IDs to localStorage, retaining them after page reload on the same device.

Implementation Boundaries

Meal recommendations use explicit filtering and sorting rules, not a trained AI model. Ingredient metadata is sample data and does not establish allergen safety. These two modifications supplement the existing menu, cart and kitchen workflow.

Challenges Faced and Verification

Implementation Challenges

- Shared state: A Context provider keeps cart quantities and favourites consistent between the menu, saved page, cart and checkout.
- Price integrity: The backend calculates prices from its catalogue instead of accepting totals supplied by the browser.
- Reliable persistence: Queued, atomic writes prevent simultaneous orders from overwriting each other in the single-process store.
- Status consistency: The kitchen cannot skip stages; the server re-checks the stage when the queued update is applied.
- Effect cleanup: Polling and feedback timers are cleared when the related components unmount.
- Responsive presentation: Mobile dish rows and reduced column counts keep controls usable on narrow screens.

Verification Results

- Production build: Vite generated the frontend bundle successfully.
- Backend tests: Seven tests passed with zero failures, including input rejection, server totals, receipt access, kitchen stage rules, recommendation constraints, restart persistence and simultaneous orders.
- Browser verification: Twenty-two checks passed on the implemented frontend and backend, with no browser runtime errors recorded.
- Viewport coverage: Desktop at 1440 × 1000 and mobile at 390 × 844; checked mobile routes had no horizontal overflow.
- Integrated flow: A submitted order appeared on the kitchen board, advanced to Ready for pickup and displayed the updated customer receipt.

Verification applies to a local, single-process prototype. Production load, complete staff authentication, every-device compatibility and real payment or messaging services were not evaluated.

Conclusion

bytemingos covers the main steps of a campus pickup order. The React components keep the pages organised, and shared state keeps the cart and favourites consistent. Express checks the order data, calculates the amount and controls each status change.

- Implemented outcomes: Menu filtering, saved favourites, a persistent cart, pickup checkout, receipt access and kitchen status management.
- Additional features: An explainable meal finder and a dedicated saved-meals page.
- Technical learning: Props, callbacks, state, effects, Context, forms, routing, class-based error handling and frontend-backend integration.
- Verification: Successful production build, backend integration tests and observed desktop/mobile ordering checks.

A production extension would require a transactional database, individual staff authentication, verified restaurant data and stock management. These additions are beyond the current mini-project scope.

GitHub Links and References

- Project source: <https://github.com/KunalKajrekar15/BYTEMINGOS>. The repository contains the React frontend, Express backend, menu assets, dependency lockfile and project reports.
- Tutorial reference implementation and image source: <https://github.com/sakshi-bisen12/Food-Delivery-App>
- React documentation: <https://react.dev/>
- Vite documentation: <https://vite.dev/>
- React Router documentation: <https://reactrouter.com/>
- Express documentation: <https://expressjs.com/>
- Node.js documentation: <https://nodejs.org/>
- CORS package: <https://github.com/expressjs/cors>

YouTube Tutorial Link

<https://www.youtube.com/watch?v=DBMPXJJfQEA>